



Universidad Tecnológica Nacional

Tecnicatura Universitaria en Programación a Distancia

Sistema de Pedidos (Food Store)

Trabajo Final de Programación 2

Grupo: 12

Integrantes:

- Lucas Matías Catanzaro
- Emiliano Fiscina
- Dante Ariel Gibbon
- Lautaro Lesniewicz

Fecha de Entrega: Junio 2026

Índice

Tema	Página
1. Marco Teórico	3
2. Decisiones Técnicas y Arquitectura	4
2.1 Organización de paquetes	5
2.2 Persistencia en memoria y baja lógica	6
2.3 Interfaz de usuario y flujo de operaciones	6
2.4 Menú principal interactivo	7
2.5 Proceso de creación de un pedido	8
3. Dificultades y Soluciones	9
4. Bibliografía y Webgrafía	10
5. Enlaces del Proyecto	10

1. Marco Teórico

Para el desarrollo de este sistema se utilizaron diversos conceptos teóricos y tecnológicos fundamentales de la programación orientada a objetos (POO) y del diseño de software.

- **Lenguaje Java:** Se seleccionó Java como lenguaje principal debido a su robustez, portabilidad y amplia utilización en el desarrollo de aplicaciones empresariales. Su paradigma orientado a objetos permite modelar entidades del mundo real de manera eficiente, facilitando la reutilización del código y el mantenimiento de la aplicación.
- **Programación Orientada a Objetos (POO):** La solución fue diseñada aplicando los principales pilares de la POO. La *Herencia* se implementó mediante la clase abstracta *Base*, que centraliza atributos comunes como identificador, fecha de creación y estado de eliminación. La *Abstracción* se utilizó a través de interfaces, como *Calculable*, permitiendo definir comportamientos generales sin depender de implementaciones específicas. El *Encapsulamiento* se aplicó mediante atributos privados y métodos de acceso controlado (getters y setters), garantizando la integridad de los datos.
- **Colecciones en Memoria:** Debido a que el proyecto no contempla la utilización de una base de datos física, se emplearon estructuras dinámicas como *ArrayList* para almacenar la información durante la ejecución del programa. Esta estrategia permitió simular operaciones CRUD (Crear, Leer, Actualizar y Eliminar) manteniendo la simplicidad requerida para el trabajo práctico.
- **Manejo de Excepciones:** Se desarrollaron excepciones personalizadas como *EntidadNoEncontradaException*, *ValidacionException* y *StockInvalidoException*. Esto permitió separar la lógica de negocio del tratamiento de errores, facilitando la detección de problemas y proporcionando mensajes claros para el usuario.
- **Arquitectura en Capas:** El sistema fue diseñado siguiendo una organización modular que separa entidades, servicios, excepciones y enumeraciones. Esta estructura mejora la mantenibilidad del código, facilita futuras ampliaciones y permite una distribución más clara de las responsabilidades.

2. Decisiones Técnicas y Arquitectura

Para el desarrollo de este sistema se optó por un enfoque basado en la separación de responsabilidades (*Separation of Concerns*), organizando la aplicación en capas lógicas independientes. Este criterio de diseño permite reducir el acoplamiento entre componentes y mejorar la cohesión interna de cada módulo.

La arquitectura implementada simula la estructura utilizada en aplicaciones empresariales reales, donde cada capa cumple una función específica. Las entidades representan los objetos del dominio del negocio, los servicios contienen las reglas de negocio y la lógica de procesamiento, mientras que las excepciones permiten gestionar errores de forma controlada.

Gracias a esta organización, el sistema resulta más sencillo de mantener, probar y ampliar. En caso de requerir futuras mejoras, como la incorporación de una base de datos o una interfaz gráfica, la estructura actual permite realizar dichas modificaciones con un impacto mínimo sobre el resto de los componentes.

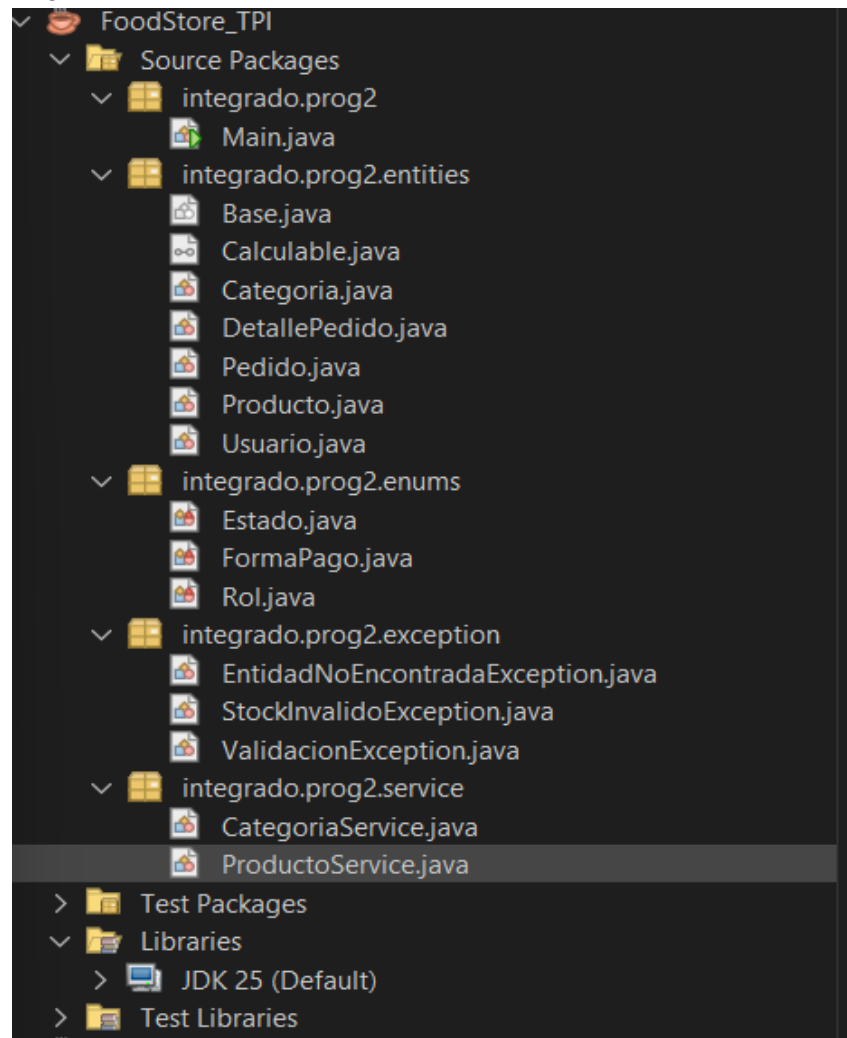
2.1 Organización de paquetes

El proyecto fue dividido en paquetes específicos, garantizando que cada clase tenga una única responsabilidad:

- **entities:** Contiene los modelos de dominio (Categoría, Producto, Usuario, Pedido y DetallePedido), encapsulando los atributos y aplicando herencia mediante la clase abstracta Base.
- **service:** Actúa como la capa de lógica de negocio y persistencia en memoria. Aquí se aplican las validaciones y se gestionan las colecciones de datos.
- **exception:** Agrupa las clases de excepciones personalizadas (ValidacionException, EntidadNoEncontradaException) para un manejo de errores limpio y controlado.
- **enums:** Centraliza los valores constantes del sistema (Estado, FormaPago, Rol).

La imagen muestra la estructura de paquetes implementada en el entorno de desarrollo.

Organización en paquetes utilizando arquitectura en capas en NetBeans



2.2 Persistencia en memoria y Baja lógica

Ante la ausencia de un motor de base de datos relacional, la persistencia de la información se resolvió mediante colecciones dinámicas (`List`) administradas por las clases de servicio. Esta decisión permitió implementar todas las funcionalidades requeridas manteniendo la simplicidad del proyecto y evitando dependencias externas.

Una de las decisiones técnicas más importantes fue la implementación de la Baja Lógica (*Soft Delete*). En lugar de eliminar físicamente los registros de las colecciones, se incorporó un atributo booleano denominado eliminado, heredado por todas las entidades desde la clase Base.

Este enfoque presenta diversas ventajas. En primer lugar, permite conservar el historial de operaciones realizadas por el sistema. Además, evita problemas de integridad referencial, especialmente en entidades relacionadas entre sí, como los pedidos y los productos. De esta manera, si un producto deja de estar disponible, los pedidos históricos continúan conservando la información original sin verse afectados.

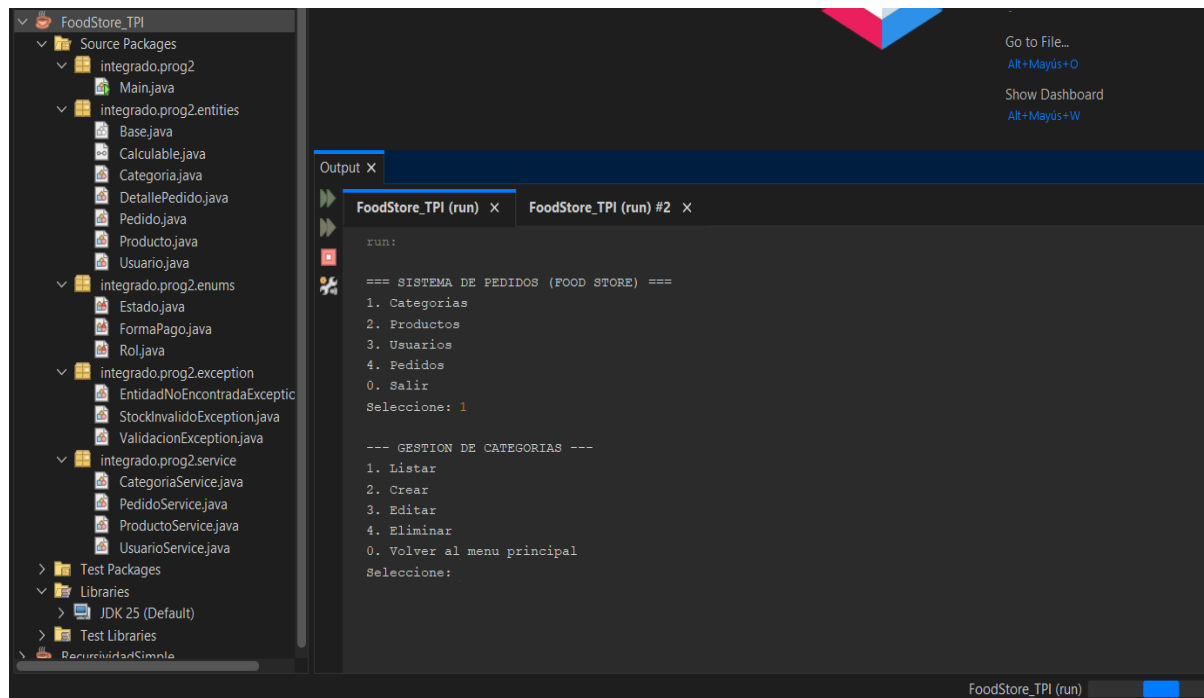
Los métodos de búsqueda y listado aplican filtros que excluyen automáticamente los registros marcados como eliminados, garantizando que el usuario visualice únicamente información activa sin comprometer la consistencia de los datos almacenados en memoria.

2.3 Interfaz de usuario y flujo de operaciones

La interacción con el usuario se gestiona íntegramente mediante una consola interactiva implementada en la clase Main. A través de menús jerárquicos, el usuario puede acceder a las distintas funcionalidades del sistema, como la administración de productos, usuarios, categorías y pedidos.

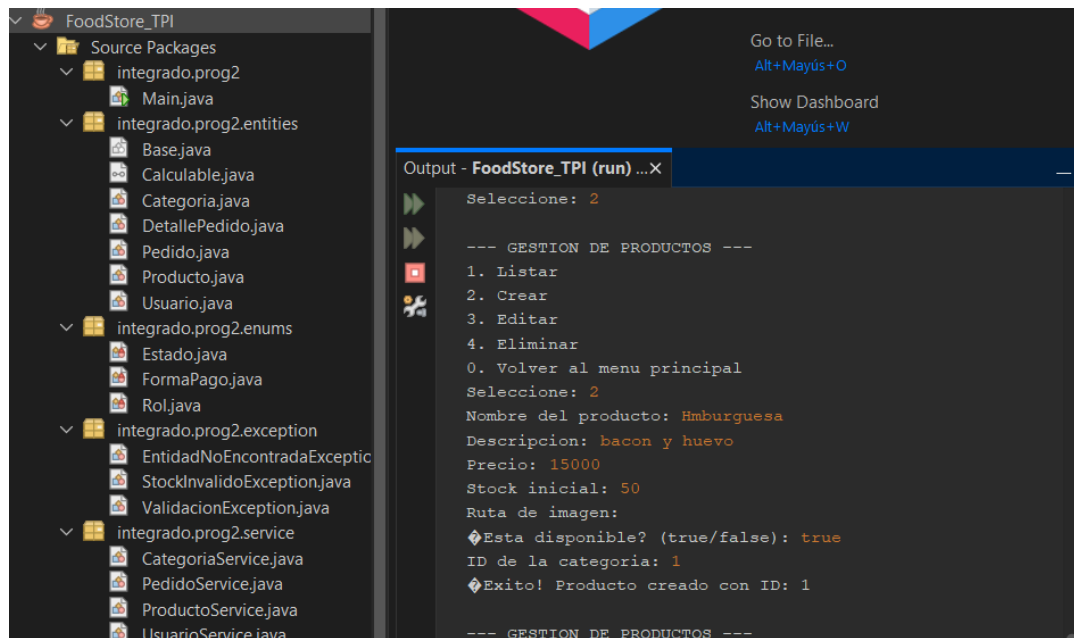
Para mejorar la experiencia de uso y aumentar la robustez de la aplicación, todas las operaciones de ingreso de datos se encuentran protegidas mediante bloques `try-catch`. Esto permite capturar errores producidos por entradas inválidas y mostrar mensajes informativos al usuario, evitando que la ejecución del programa finalice inesperadamente.

El flujo general de operaciones fue diseñado para ser intuitivo, guiando al usuario paso a paso en cada proceso y validando la información ingresada antes de ejecutar cualquier acción sobre los datos almacenados.



2.4 Menú principal interactivo

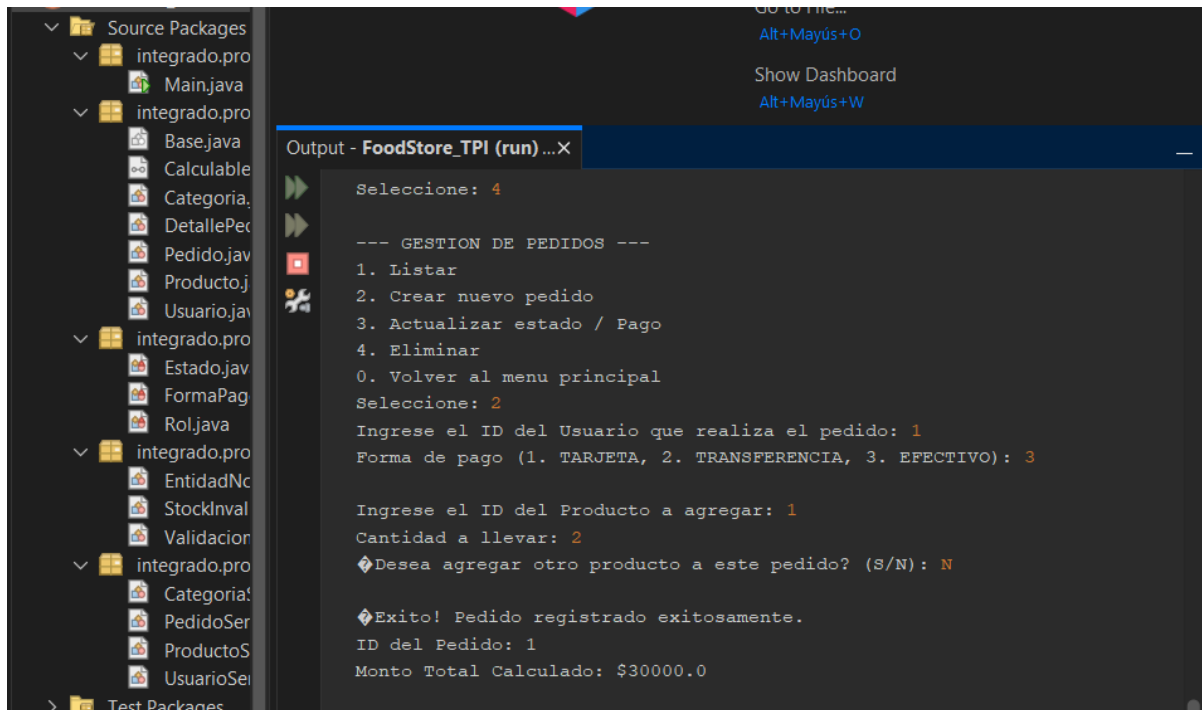
El menú interactivo permite navegar de forma fluida entre las diferentes opciones del sistema de gestión, aislando las operaciones de terminal y garantizando un control estructurado de la ejecución.



2.5 Proceso de creación de un pedido

El proceso de creación de un pedido se realiza descontando adecuadamente el stock disponible de los productos seleccionados.

El núcleo transaccional del sistema reside en el módulo de Pedidos. Se implementó un bucle interno que simula un "carrito de compras", permitiendo agregar múltiples instancias de `DetallePedido` a un `Pedido` central. Durante este proceso, el sistema verifica las reglas de negocio, como la disponibilidad de stock en tiempo real y la validación del usuario comprador.



```
Source Packages
├── integrado.pro
│   ├── Main.java
│   ├── Base.java
│   ├── Calculable
│   ├── Categoria
│   ├── DetallePec
│   ├── Pedido.java
│   ├── Producto.j
│   └── Usuario.jav
├── integrado.pro
│   ├── Estado.jav
│   ├── FormaPag
│   └── Rol.java
├── integrado.pro
│   ├── EntidadNc
│   ├── StockInval
│   └── Validacion
├── integrado.pro
│   ├── Categoria!
│   ├── PedidoSer
│   ├── ProductoS
│   └── UsuarioSei
└── Test Packages

Output - FoodStore_TPI (run) ...X
Seleccione: 4

--- GESTION DE PEDIDOS ---
1. Listar
2. Crear nuevo pedido
3. Actualizar estado / Pago
4. Eliminar
0. Volver al menu principal
Seleccione: 2
Ingrese el ID del Usuario que realiza el pedido: 1
Forma de pago (1. TARJETA, 2. TRANSFERENCIA, 3. EFECTIVO): 3

Ingrese el ID del Producto a agregar: 1
Cantidad a llevar: 2
❖Desea agregar otro producto a este pedido? (S/N): N

❖Exito! Pedido registrado exitosamente.
ID del Pedido: 1
Monto Total Calculado: $300000.0
```

Se listan pedidos generados mostrando el desglose de productos y la sumatoria total.

Finalmente, para estandarizar el procesamiento de montos, se aplicó la interfaz `Calculable`. Esto obliga a la entidad `Pedido` a recalcular su atributo `total` sumando automáticamente los subtotales de sus detalles, previniendo inconsistencias matemáticas que podrían surgir de una carga manual.

3. Dificultades y Soluciones

Durante el desarrollo del proyecto surgieron distintos desafíos relacionados con el modelado de entidades, la validación de datos y la gestión de relaciones entre objetos. Cada uno de estos inconvenientes fue abordado mediante soluciones orientadas a mantener la integridad del sistema y garantizar una experiencia de usuario estable y consistente.

Dificultad	Solución
Manejo de relaciones 1 a N entre el Pedido y sus Detalles dentro de la consola interactiva.	Se implementó un bucle <code>while</code> en el Main que permite agregar múltiples detalles a la lista interna del pedido antes de guardarlo definitivamente a través de <code>PedidoService</code> .
Evitar excepciones del sistema (como <code>NumberFormatException</code>) cuando el usuario ingresa texto en lugar de números en los menús.	Se envolvió la lectura del menú en bloques <code>try-catch</code> , parseando <code>scanner.nextLine()</code> para evitar saltos de línea huérfanos y controlando el error al mostrar un mensaje amigable al usuario.
Garantizar que los elementos eliminados lógicamente (Baja Lógica) no interfieran en las transacciones futuras, como asignar un producto eliminado a un nuevo pedido.	Se incluyeron validaciones estrictas en la capa de Servicios utilizando la condición <code>!entidad.isEliminado()</code> . De esta forma, los métodos de búsqueda y listado filtran dinámicamente los registros, protegiendo la integridad del sistema sin necesidad de borrar los datos en memoria.

4. Bibliografía y Webgrafía

- ---

Oracle. (n.d.). *Java Documentation*. Recuperado de <https://docs.oracle.com/en/java/>
- Material de estudio de la cátedra de Programación 2.

5. Enlaces del Proyecto

- ---

Repositorio GitHub: https://github.com/betsmatias/sistema_pedidos_food_store.git
- **Video Demostrativo:** <https://youtu.be/UEHEakKNqDQ>